

LP III - Procesos en Linux

Ejercicios de Repaso



```
$ gcc -Wall -o identidad identidad.c  
$ ./identidad  
[main] PID=4021 PPID=3980  
[hijo] PID=4022, duermo 2s...  
[padre] el hijo salio con codigo 7
```

Ejercicios de repaso

1

Identidad de un proceso

`fork()`, `getpid/getppid`, `wait()` básico

2

Mini ejecutor de comandos

El patrón `fork` + `exec (spawn)`, `waitpid()`

3

Manejador de señales

`sigaction()`, `sig_atomic_t`, señales propias

4

Horda de zombies

Procesos zombie, `SIGCHLD`, limpieza asíncrona

Identidad de un proceso

1. Al iniciar el programa, imprimir el PID propio y el PPID (proceso padre).
2. Crear un proceso hijo con `fork()`.
3. El proceso hijo debe mostrar su identidad, dormir 2 segundos y terminar con `exit(7)`.
4. El padre debe esperar por el proceso hijo con `wait()` y mostrar su código de salida.

Identidad de un proceso

identidad.c

```
// gcc -Wall -o identidad identidad.c && ./identidad

#include <stdio.h>
#include <sys/types.h>
#include <unistd.h>
#include <sys/wait.h>

int main(void) {
    printf("[main] PID=%d PPID=%d\n", (int) getpid(), (int) getppid());
    pid_t pid = fork();
    if (pid == 0) {
        printf("[hijo] PID=%d, duermo 2s...\n", (int) getpid());
        sleep(2);
        exit(7);
    }
    int status;
    wait(&status);
    if (WIFEXITED(status))
        printf("[padre] el hijo retornó con código %d\n", WEXITSTATUS(status));
    return 0;
}
```

Mini ejecutor de comandos

1. El programa debe recibir un comando a ejecutar junto con sus argumentos por línea de comandos (ej: ./ejecutar ls -l /tmp).
2. El programa debe crear un proceso hijo con `fork()`.
3. El proceso hijo debe ejecutar el comando recibido con `execvp()` y capturar su código de salida.
4. Si `execvp()` falla, el proceso hijo debe informar del error y terminar apropiadamente.
5. El proceso padre debe esperar por el proceso hijo con `waitpid()` y mostrar el código de salida del commando ejecutado.

Mini ejecutor de comandos

[ejecutar.c](#)

```
// gcc -Wall -o ejecutar ejecutar.c && ./ejecutar ls -l

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char** argv) {
    if (argc < 2) { fprintf(stderr, "Uso: %s <cmd> [args...]\n", argv[0]); exit(1); }
    pid_t pid = fork();
    if (pid == 0) {
        execvp(argv[1], &argv[1]);
        perror("execvp");
        _exit(127);
    }
    int status;
    waitpid(pid, &status, 0);
    if (WIFEXITED(status))
        printf("\n%s' termino con codigo %d\n", argv[1], WEXITSTATUS(status));
    return 0;
}
```

Manejador de señales

1. Escriba un programa que instale un manejador para SIGINT (Ctrl+C) que incremente un contador, sin terminar el programa.
2. Asimismo, que instale un manejador para SIGUSR1 que responda al pedido de "mostrar el PID".
3. En un bucle principal, imprimir un contador de segundos y asimismo, atender el pedido de PID cuando corresponda.
4. Terminar el programa de forma ordenada cuando el mismo recibe la señal SIGINT por tercera vez.

Manejador de señales

manejador.c

```
// gcc -Wall -o manejador manejador.c && ./manejador
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
volatile sig_atomic_t interrupciones = 0, pedido_pid = 0;
void manejar_sigint(int signum) { interrupciones++; }
void manejar_sigusr1(int signum) { pedido_pid = 1; }

int main(void) {
    struct sigaction sa = {0};
    sa.sa_handler = manejar_sigint;
    sigaction(SIGINT, &sa, NULL);
    sa.sa_handler = manejar_sigusr1;
    sigaction(SIGUSR1, &sa, NULL);
    printf("PID=%d - Ctrl+C x3 sale, USR1 saluda\n", (int)getpid());
    while (interrupciones < 3) {
        if (pedido_pid) { printf("Hola, soy %d\n", (int)getpid()); pedido_pid = 0; }
        printf("trabajando... (Ctrl+C: %d/3)\n", interrupciones);
        sleep(1);
    }
    printf("Recibi %d interrupciones. Chau.\n", interrupciones);
    return 0;
}
```


Horda de zombies

Parte A — Observar el problema

- Crear 5 procesos hijos con `fork()` en un bucle; cada hijo termina de inmediato.
- El padre NO llama a `wait()`: se queda dormido 15 segundos.
- Mientras duerme, observar los procesos zombie con `ps` en otra terminal.

Parte B — Resolverlo

- Instalar un manejador de `SIGCHLD` que limpie a los hijos automáticamente.
- Usar `waitpid(-1, &status, WNOHANG)` dentro de un `while` en el manejador.
- Verificar con `ps` que ya no quedan procesos `<defunct>`.

Horda de zombies – Caso A

zombies.c

```
// gcc -Wall -o zombies zombies.c && ./zombies

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(void) {
    for (int i = 0; i < 5; i++)
        if (fork() == 0) { printf("[hijo %d] PID=%d\n", i, (int)getpid()); exit(0); }

    printf("[padre] duermo 15s SIN wait(). Corran: ps -e -o pid,ppid,stat,cmd\n");
    sleep(15);
    printf("[padre] termino.\n");
    return 0;
}
```

Deberían ver `STAT=Z` y `"<defunct>"` en varias líneas de la salida de `ps`.

Horda de zombies – Caso B

zombies_ok.c

```
// gcc -Wall -o zombies_ok zombies_ok.c && ./zombies_ok

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>

void limpiar(int signum) {
    int status;
    while (waitpid(-1, &status, WNOHANG) > 0) { }
}

int main(void) {
    struct sigaction sa = {0};
    sa.sa_handler = limpiar;
    sigaction(SIGCHLD, &sa, NULL);
    for (int i = 0; i < 5; i++)
        if (fork() == 0) { printf("[hijo %d] PID=%d\n", i, (int)getpid()); exit(0); }
    printf("[padre] limpieza activa. ps -e -o pid,ppid,stat,cmd no debe mostrar zombies.\n");
    sleep(15);
    printf("[padre] termino.\n");
    return 0;
}
```